



Unidade 4 — Por que desenvolver seu próprio Framework?



Roteiro da Aula (50 min)

Tempo	Atividade
0-5 min	Abertura e contextualização
5-15 min	Revisão da arquitetura do mini framework
15-25 min	Discussão: o que o Express faz que nós não fazemos?
25-40 min	Demonstração prática: construir o framework
40-47 min	Desafio: adicionar funcionalidade
47-50 min	Síntese e orientação da atividade



0-5 min: Contextualização

O que estudamos na Unidade 4:

- Aula 19: Por que criar um Framework?
- Aula 20: Vantagens e desvantagens
- Aula 21: Arquitetura de um Framework
- Aula 22: Construindo um Router
- Aula 23: Middleware e Pipeline
- Aula 24: Do Mini Framework ao Framework Real

O que vamos fazer hoje:

Construir o mini framework completo, do zero, em tempo real — e testar com uma aplicação.



5-15 min: Revisão da Arquitetura

```
Requisição HTTP
↓
Enhance req/res (json, status, params)
↓
Middleware Pipeline (logger, jsonParse)
↓
Router (match method + URL → handler)
↓
Handler (controller)
↓
Resposta HTTP
```



25-40 min: Construção do Framework

Arquivo: `mini-framework.js`

```
const http = require('http');

function MiniFramework() {
  const routes = [];
  const middlewares = [];

  // Registrar middleware
  function use(fn) {
    middlewares.push(fn);
  }

  // Registrar rota
  function addRoute(method, path, handler) {
    const paramNames = [];
    const pattern = path.replace(/:([w+])/g, (_, name) => {
      paramNames.push(name);
      return '([/]+)';
    });
    routes.push({
      method: method.toUpperCase(),
      handler,
      paramNames,
      regex: new RegExp(`^${pattern}$`)
    });
  }

  // Métodos HTTP
  function get(path, handler) { addRoute('GET', path, handler); }
  function post(path, handler) { addRoute('POST', path, handler); }
  function put(path, handler) { addRoute('PUT', path, handler); }
  function del(path, handler) { addRoute('DELETE', path, handler); }

  // Executar pipeline
  function runPipeline(req, res, done) {
    let i = 0;
    function next() {
      if (i < middlewares.length) {
        middlewares[i++](req, res, next);
      } else {
        done();
      }
    }
    next();
  }

  // Servidor
  function listen(port, cb) {
    const server = http.createServer((req, res) => {
      // Enhance res
      res.json = (data) => {
        res.setHeader('Content-Type', 'application/json');
        res.end(JSON.stringify(data));
      };
      res.status = (code) => { res.statusCode = code; return res; };

      // Parse URL
      const [urlPath] = req.url.split('?');

      // Executar middlewares - depois rotar
      runPipeline(req, res, () => {
        for (const route of routes) {
          if (route.method !== req.method) continue;
          const match = urlPath.match(route.regex);
          if (match) {
            req.params = {};
            route.paramNames.forEach((name, i) => {
              req.params[name] = match[i + 1];
            });
            return route.handler(req, res);
          }
        }
        res.statusCode = 404;
        res.json({ erro: 'Rota não encontrada' });
      });
    });
    server.listen(port, cb);
  }

  return { use, get, post, put, delete: del, listen };
}
```





Demonstração: Usando o Framework

```
const MiniFramework = require('./mini-framework');
const app = MiniFramework();

// Middleware de log
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

// Middleware de JSON
app.use((req, res, next) => {
  if (req.headers['content-type'] === 'application/json') {
    let body = '';
    req.on('data', chunk => body += chunk);
    req.on('end', () => {
      try { req.body = JSON.parse(body); } catch (e) { req.body = {}; }
      next();
    });
  } else { next(); }
});

let usuarios = [{ id: 1, nome: 'Maria' }];

app.get('/api/usuarios', (req, res) => {
  res.json(usuarios);
});

app.post('/api/usuarios', (req, res) => {
  req.body.id = usuarios.length + 1;
  usuarios.push(req.body);
  res.status(201).json(req.body);
});

app.listen(3000, () => console.log('Mini Framework na porta 3000'));
```





40–47 min: Desafio

Desafio: Adicione uma funcionalidade ao mini framework.

Opções:

1. **Middleware de erro:** capturar erros e devolver 500;
2. **Query strings:** extrair `?page=1&limit=10` de `req.url`;
3. **Rota PUT:** atualizar um recurso existente;
4. **Rota DELETE:** remover um recurso.



47–50 min: Síntese

O que construímos hoje:

- Um **mini framework** funcional com ~80 linhas;
- **Router** com parâmetros de URL;
- **Middleware Pipeline** com next();
- **res.json()** e **res.status()**;
- **Servidor HTTP** integrado.

Conclusão:

Frameworks não são mágica. São **código organizado** que resolve **problemas comuns**.

Atividade Prática Orientada: disponível na pasta da Unidade 4.

